

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Base de Datos II — Unidad 2: Optimización de Consultas

Estudiantes: Facundo Deseff, Facundo Ramirez

Repositorio: https://github.com/FacundoDeseff/Tps_BasedeDatos.git

Trabajo Práctico — Semana 3

Optimización asistida por IA sobre Food Store: filtros, planes de ejecución e índices

Esta práctica aplica los contenidos de la Semana 3 sobre el proyecto integrador Food Store, con la inteligencia artificial como motor primario del trabajo, tal como lo establece la cátedra. El objetivo no es que la IA optimice sola, sino que el estudiante mida, decida y justifique cada cambio con evidencia de EXPLAIN ANALYZE — la IA propone, el estudiante verifica.

Objetivos de la práctica

- Analizar consultas lentas sobre una base Food Store poblada masivamente y optimizarlas midiendo la mejora real.
- Usar la IA (OpenCode / Kiro) como motor primario para proponer reescrituras e índices a partir de un plan de ejecución real.
- Contrastar la explicación de un plan generada por IA contra el plan real, detectando imprecisiones.
- Generar consultas resumen y subconsultas bajo especificación precisa, verificando la equivalencia de resultados contra una alternativa propia.
- Documentar cada decisión —aceptada o descartada— en la bitácora de uso de IA (DUIA).

Requisitos previos

- Base Food Store creada a partir de schema_completo.sql, con el seed de data.sql aplicado.
- Repositorio Git de la cátedra con OpenCode y Kiro configurados, según el protocolo de seguridad: copia, transacción, respaldo.
- Acceso a psql / DBeaver / pgAdmin para ejecutar EXPLAIN ANALYZE.

Parte 1 — Poblar la base masivamente con datos generados por IA

Antes de poder medir una optimización, hace falta un volumen de datos realista. Los seeds de la Guía Docente cargan apenas un puñado de filas; para que un Seq Scan sea lento hace falta escala.

1. Pedir a la IA (OpenCode) un script SQL o un generador (por ejemplo, con generate_series) que cargue: al menos 50.000 productos distribuidos en las categorías existentes, 20.000 usuarios y 200.000 pedidos con sus detalles.

2. Leer línea por línea el script generado antes de ejecutarlo: verificar que respeta las restricciones (CHECK, UNIQUE, FK) y que no toca la base de producción.
3. Ejecutarlo dentro de una transacción, sobre una copia de la base del proyecto, con respaldo previo — protocolo de seguridad de la cátedra.
4. Correr ANALYZE sobre las tablas afectadas para que el optimizador actualice sus estadísticas antes de medir.

```
-- Ejemplo de prompt / spec para la IA (a completar y ajustar por el estudiante):  
-- "Generá un script SQL para PostgreSQL que inserte 50.000 filas en producto,  
-- distribuidas de forma pareja entre las categorías existentes, con precios  
-- entre 500 y 5000 y stock aleatorio entre 0 y 200. Usá generate_series y  
-- no uses PL/pgSQL si no es necesario. No modifiques ninguna otra tabla."
```

Parte 2 — Laboratorio: consultas lentas, EXPLAIN y optimización medida

Con la base ya poblada masivamente, se eligen consultas reales de queries.sql que se vuelven lentas a esta escala — por ejemplo, el listado de productos por categoría o el historial de pedidos por usuario sin índice — y se optimizan siguiendo el flujo profesional de la cátedra: medir antes, proponer, medir después, decidir con datos.

2.1 Consigna

1. Elegir al menos tres consultas de queries.sql (o *variantes propias sobre el modelo*) que tarden de forma perceptible sobre la base masiva.
2. Para cada una, correr EXPLAIN ANALYZE y guardar el plan completo (*captura o texto*) antes de cualquier cambio.
3. Pasarle a la IA el plan real obtenido y pedirle que proponga reescrituras y/o índices que lo mejoren, justificando cada propuesta en términos del propio plan (qué nodo ataca, por qué esperaría que cambie).
4. Leer línea por línea cada CREATE INDEX o reescritura propuesta antes de aplicarla. Rechazar toda propuesta que no se entienda por completo.
5. Aplicar los cambios aceptados sobre la copia de trabajo y volver a medir con EXPLAIN ANALYZE.
6. Completar la tabla comparativa de la sección 2.2 para cada consulta.

2.2 Tabla de resultados (incluir en la entrega, una fila por consulta)

Consulta	Plan antes (nodo, cost, tiempo real)	Cambio aplicado	Plan después (nodo, cost, tiempo real)	Mejora
<u>Listado de productos por categoría</u> SELECT * FROM producto WHERE id_categoria = 3 AND eliminado = FALSE;	<u>Seq Scan on producto</u> cost: 0.00..1140.00 tiempo: 2.924 ms	CREATE INDEX idx_producto_categoria_vig ON producto (id_categoria) WHERE eliminado = FALSE;	<u>Index Scan using idx_producto_categoria_vig</u> cost: 0.29..4.31 tiempo: 0.100 ms	<u>~29.2x (de 2.92 ms a 0.10 ms)</u>
<u>Historial de pedidos por usuario</u> SELECT * FROM pedido WHERE usuario_id = 4520;	<u>Seq Scan on pedido</u> cost: 0.00..5442.00 tiempo: 9.001 ms (199.987 filas descartadas)	CREATE INDEX idx_pedido_usuario ON pedido (usuario_id);	<u>Bitmap Index Scan + Heap Scan</u> cost: 4.37..42.75 tiempo: 0.099 ms	<u>~90.9x (de 9.00 ms a 0.09 ms)</u>
<u>Facturación por categoría</u> SELECT pr.id_categoria, SUM(dp.subtotal) FROM detalle_pedido dp JOIN producto pr ON pr.id_producto = dp.producto_id WHERE dp.eliminado = FALSE GROUP BY pr.id_categoria;	<u>Finalize GroupAggregate (Parallel Seq Scan)</u> cost: 6380.58..6380.72 tiempo: 47.270 ms	CREATE INDEX idx_detalle_producto ON detalle_pedido (producto_id, subtotal) WHERE eliminado = FALSE;	<u>Finalize GroupAggregate</u> cost: 6380.58..6380.72 tiempo: 44.827 ms	<u>~1.05x (acotada por agregación masiva)</u>

Criterio de aceptación

Ninguna propuesta de la IA se aplica “porque lo dijo la IA”: se aplica solo si el estudiante puede explicar, en la defensa oral, por qué el índice o la reescritura deberían mejorar ese plan en particular, y la medición posterior lo confirma. Si una propuesta no mejora el tiempo real, se documenta igual —qué se esperaba, qué pasó, por qué— en lugar de descartarla en silencio.

Parte 3 — Lectura crítica de planes interpretados por IA

Un asistente de IA puede explicar en lenguaje natural qué hace un plan de EXPLAIN ANALYZE, pero esa explicación puede contener imprecisiones —por ejemplo, confundir un costo estimado con un tiempo real, o atribuir la mejora a la causa equivocada—. Esta parte entrena la lectura crítica de esas explicaciones.

1. Tomar uno de los planes reales obtenidos en la Parte 2 (con índice ya aplicado).
2. Pedirle a la IA que lo explique en lenguaje natural, nodo por nodo, sin mostrarle más contexto que el texto del plan.
3. Contrastar esa explicación contra el plan real, frase por frase, y marcar cualquier afirmación incorrecta o imprecisa (por ejemplo: “confunde cost con milisegundos”, “atribuye la mejora al índice equivocado”, “ignora el Rows Removed by Filter”).
4. Documentar los hallazgos en la tabla siguiente.

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
"El costo inicial de 4.37 indica que la consulta tardó 4.37 milisegundos en arrancar en el motor."	No	Confusión de unidades. Las unidades de cost son estimaciones abstractas del planificador relacional sobre uso de disco y CPU. El tiempo real medido por el motor fue de actual time=0.037..0.088 ms y Execution Time: 0.099 ms.
"El optimizador seleccionó un Bitmap Index Scan porque la columna usuario_id cuenta con un índice B-tree, evitando escanear las 200.000 filas de pedido."	Sí	El optimizador detectó que era más eficiente mapear las tuplas coincidentes en un mapa de bits en memoria (Bitmap Index Scan) y acceder ordenadamente a los bloques de disco (Bitmap Heap Scan), reduciendo I/O aleatorio.
"El motor descartó 199.987 filas mediante la cláusula Rows Removed by Filter en el plan optimizado."	No	Ese descarte masivo ocurrió únicamente en el Seq Scan inicial previo a los índices. En el plan optimizado el motor accedió directamente a las páginas necesarias (Heap Blocks: exact=13), sin evaluar las tuplas restantes.

Parte 4 — Consultas resumen y subconsultas bajo especificación precisa

Se trata de practicar el flujo de especificar con precisión —en lugar de pedir “hacé una consulta que...” de forma vaga— y de verificar que lo que la IA generó es equivalente a lo que se pidió, no solo que “corre sin error”.

1. Redactar una especificación precisa (spec) para al menos dos consultas sobre Food Store: una de resumen (agregación) y una con subconsulta. La especificación debe fijar explícitamente: tablas involucradas, filtro de borrado lógico, columnas de salida, orden y, si corresponde, criterio de corte (LIMIT, HAVING).
2. Pedirle a la IA que genere el SQL a partir de esa especificación, sin mostrarle una solución previa.
3. Escribir, de forma independiente, una segunda versión propia (o pedirle a la IA una alternativa con distinta estructura: subconsulta vs. join, por ejemplo) que responda la misma pregunta.
4. Verificar la equivalencia de resultados entre ambas versiones — por ejemplo, comparando el conteo de filas y una muestra de valores, o con EXCEPT entre ambos resultados.

Desarrollo Parte 4 — Especificación precisa y equivalencia formal

Especificación precisa (Spec):

"Generaré una consulta SQL sobre el esquema de Food Store que devuelva el identificador, la fecha y el importe total de todos los pedidos vigentes (**eliminado = FALSE**) cuyo total sea estrictamente superior al promedio general de compras de pedidos no eliminados. Ordená de mayor a menor por total."

Variante A (Subconsulta escalar en cláusula WHERE):

SQL

```
SELECT id_pedido, fecha, total
FROM pedido
WHERE eliminado = FALSE
      AND total > (SELECT AVG(total) FROM pedido WHERE eliminado = FALSE)
ORDER BY total DESC;
```

Variante B (CTE agregada con producto cartesiano):

SQL

```
WITH promedio AS (  
    SELECT AVG(total) AS prom  
    FROM pedido  
    WHERE eliminado = FALSE  
)  
SELECT p.id_pedido, p.fecha, p.total  
FROM pedido p, promedio  
WHERE p.eliminado = FALSE  
    AND p.total > promedio.prom  
ORDER BY p.total DESC;
```

Comprobación formal de equivalencia: Se contrastaron ambas variantes mediante resta de conjuntos con el operador EXCEPT en PostgreSQL:

```
1  (  
2      SELECT id_pedido, fecha, total  
3      FROM pedido  
4      WHERE eliminado = FALSE AND total > (SELECT AVG(total) FROM pedido WHERE eliminado = FALSE)  
5  )  
6  EXCEPT  
7  (  
8      WITH promedio AS (SELECT AVG(total) AS prom FROM pedido WHERE eliminado = FALSE)  
9      SELECT p.id_pedido, p.fecha, p.total  
10     FROM pedido p, promedio  
11     WHERE p.eliminado = FALSE AND p.total > promedio.prom  
12 );
```

Resultado obtenido: (0 rows). La diferencia de conjuntos es estrictamente vacía, demostrando que ambas consultas devuelven exactamente las mismas tuplas sin inconsistencias semánticas ni pérdida de filas por nulos.

```
-- Verificación de equivalencia entre dos consultas que deberían dar el mismo resultado
(consulta_A) EXCEPT (consulta_B); -- filas en A que no están en B
(consulta_B) EXCEPT (consulta_A); -- filas en B que no están en A
-- Ambas deben devolver 0 filas si son equivalentes
```

Ejemplo de especificación precisa para pedirle a la IA (a adaptar y completar por el estudiante, no copiar literal):

Spec de ejemplo

«Genera una consulta SQL sobre el esquema de Food Store que devuelva, para cada categoría vigente (eliminado = FALSE), el nombre de la categoría y la cantidad de productos vigentes que tiene, incluyendo las categorías sin productos vigentes con cantidad 0. Ordená de mayor a menor cantidad. No uses SELECT *.»

Parte 5 — Competencia de optimización entre equipos

Cierre de la práctica: todos los equipos reciben la misma consulta lenta —fijada por la cátedra sobre la base masiva común— y compiten por lograr el mejor plan posible, con la IA como asistente de cada equipo.

- Consigna común: se entrega una única consulta lenta (por ejemplo, un listado de productos por categoría con filtro de precio y orden, sin índice) igual para todos los equipos.
- Cada equipo usa su asistente de IA para proponer reescrituras e índices, pero decide y aplica solo lo que puede justificar.
- Gana el equipo con el mejor tiempo de ejecución real (EXPLAIN ANALYZE), no el mejor costo estimado.
- Cada equipo documenta en su bitácora qué probó, qué descartó y por qué, incluso las propuestas de la IA que no funcionaron.

Registro de la competencia

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
Grupo 4	Índice B-tree compuesto y parcial sobre producto: CREATE INDEX idx_comp_prod ON producto(id_categoria, precio_lista) WHERE eliminado = FALSE;	22.40 ms	0.31 ms	~72.2x

Declaración de Uso de IA (DUIA)

Obligatoria para toda la práctica, según la política de uso responsable de IA de la cátedra. Completar una fila por cada uso relevante de IA durante el trabajo.

Herramienta	Para qué se usó	Prompt / spec (resumen)	Se aceptó / se descartó — por qué
Gemini	Asistencia en la generación del script de carga masiva DML y reescritura de variantes para la prueba de equivalencia.	"Generar bloque transaccional atómico en PostgreSQL para poblar 50k productos, 20k usuarios y 200k pedidos respetando claves foráneas; formular reescritura con CTE para comprobar equivalencia con EXCEPT."	Se aceptó parcialmente. Se tomó el uso de generate_series y la transacción atómica BEGIN . . . COMMIT. Se modificó la asignación aleatoria de IDs para que respete los rangos reales de las claves foráneas y se recalculó el total de los pedidos con agregación de subtotales para evitar inconsistencias de datos.
Gemini	Análisis de diagnósticos con EXPLAIN ANALYZE, propuesta de estrategias de indexación y explicación de nodos de escaneo.	"Interpretar el plan de ejecución de 200k filas con Seq Scan, sugerir índices mínimos óptimos y explicar la transición a Bitmap Index Scan."	Se aceptó la creación del índice B-tree sobre pedido(usuario_id). Se descartó una recomendación de crear un índice simple sobre la columna eliminado, adaptándola a un índice parcial (WHERE eliminado = FALSE) para ahorrar almacenamiento y selectividad. Se refutó su explicación teórica al advertir que confundió las unidades de costo del planificador con tiempo real en milisegundos.

Entregables

- Script de carga masiva (leído, probado y ejecutado bajo el protocolo de seguridad).
- Tabla comparativa de la Parte 2, con capturas o texto de los planes antes/después.
- Tabla de lectura crítica de la Parte 3.
- Las dos consultas de la Parte 4 (spec + SQL generado + verificación de equivalencia).
- Registro de la competencia de la Parte 5.
- Declaración de Uso de IA (DUIA) completa.
- Defensa oral: cualquier script o índice de la entrega debe poder explicarse en el momento.

Criterios de evaluación

Dimensión	Qué se evalúa	Peso
Medición	EXPLAIN ANALYZE correcto y completo, antes y después, para cada cambio	30%
Criterio de aceptación	Las propuestas de la IA se validan, no se aplican a ciegas	25%
Lectura crítica	Se detectan imprecisiones reales en las explicaciones generadas por IA	20%
Equivalencia verificada	Las consultas alternativas se comprueban formalmente equivalentes	15%
Bitácora y defensa	DUIA completa y capacidad de explicar cada decisión	10%